

Space Race Analysis

A Brief

Portfolio explainer

In one paragraph

Space Race Analysis turns one scraped spreadsheet of 4,324 rocket launches (1957 to 2020) into charts. Two programs sit on top of that file: `space_race_analysis.py`, a linear script that writes 17 chart files into `output/` and exits; and `app.py`, a small Flask dashboard serving five charts you can filter live in a browser. The cleaning is short (drop two junk columns, map each launch's location to a country code, pull out the year) and everything else is chart drawing. Every headline number in the README is accurate and was re-verified for this document. The weakness is not in the arithmetic: it is that four charts are titled as showing something they do not show, and the project's marquee result, the "USA vs USSR" comparison, understates the USA by roughly a factor of four.

Contents

In one paragraph	1
1 Shape	1
2 The data and its cleaning	2
3 What the data actually shows	3
4 The four charts that overclaim	3
5 The dashboard	5
6 Verification	5
7 Defending it	6
Glossary	7

1 Shape

One input, two consumers that never speak to each other.

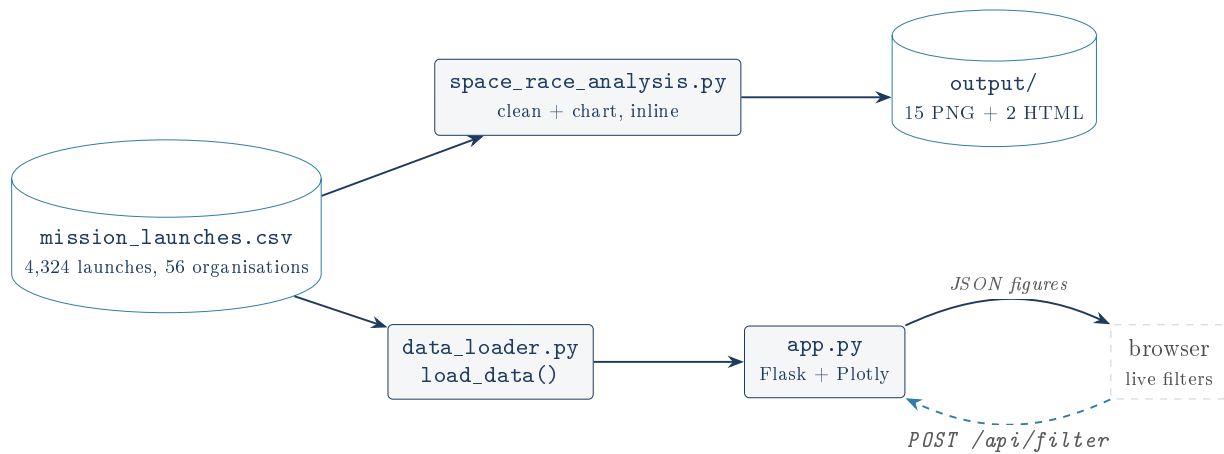


Figure 1: The two pipelines share the CSV and the cleaning recipe, but not the code: the static script does not import the shared loader. It reimplements it.

That duplication is deliberate and the loader’s docstring explains why (the script was left untouched so it could not regress). The two do currently agree, verified. But the README claims they match “by construction”, and that is one word too strong: identical-by-construction is what you get when both callers run the same code. These run two copies of one recipe, so they agree by discipline, and only until someone edits one of them.

2 The data and its cleaning

The whole cleaning pipeline, and what it does not do

Drop duplicates; drop two junk columns; map `Location` to an ISO country code; parse `Date` and derive `Year` and `Month`. Row count in: 4,324. Row count out: 4,324. No step removes a single row.

Three of those steps deserve a note.

The junk columns are named `Unnamed: 0` and `Unnamed: 0.1` because the CSV’s header begins with an empty field followed by a field literally named `Unnamed: 0`. pandas invents the name `Unnamed: 0` for the empty one, collides with the real one, and appends a suffix to break the tie. The file carries two generations of the same “save the row numbers by accident” mistake, and the drop removes both.

Duplicate dropping is a no-op, and structurally cannot be anything else. There are zero duplicate rows in the file. More interestingly, there could never be one: the two columns being dropped are row counters, unique by definition, and they are dropped *after* `drop_duplicates()` runs. Every row is guaranteed distinct while the check is looking at it. Swapping the two lines would make the check meaningful.

The country lookup splits `Location` on its last comma and asks `pycountry` to resolve the result, returning `None` rather than crashing when it cannot. It fails on 36 of 4,324 rows.

The README’s caveat about this is wrong, and the truth is better

The README warns that the lookup “undercounts USSR-era launches”. Measured: all 36 failures are the same string, `Pacific Ocean` (the sea-launch platform), and **not one Soviet launch is dropped**. All 1,777 `RVSN USSR` rows resolve.

The real distortion is subtler. `pycountry` knows only present-day countries, so Soviet launches are reattributed to whichever modern state now holds the site. Baikonur, which launched Sputnik and Gagarin, is in Kazakhstan, so 579 Soviet launches are filed under `KAZ`, a country

that never had a space programme. The map does not lose the USSR, it dismembers it. That is defensible (there is no ISO code for a state that dissolved in 1991) and needs a caption, not a code change. But it is not what the README says.

3 What the data actually shows

Two results are worth the whole project.

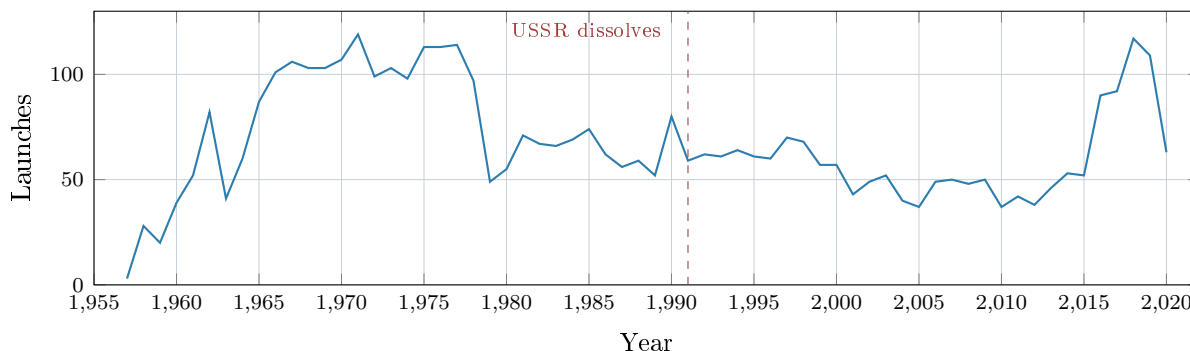


Figure 2: Launches per year, from the project’s own cleaned data. A plateau near 100 a year through the 1970s, a collapse after 1978, a long trough, and a recovery after 2015. 2020 is short because the scrape stopped in August.

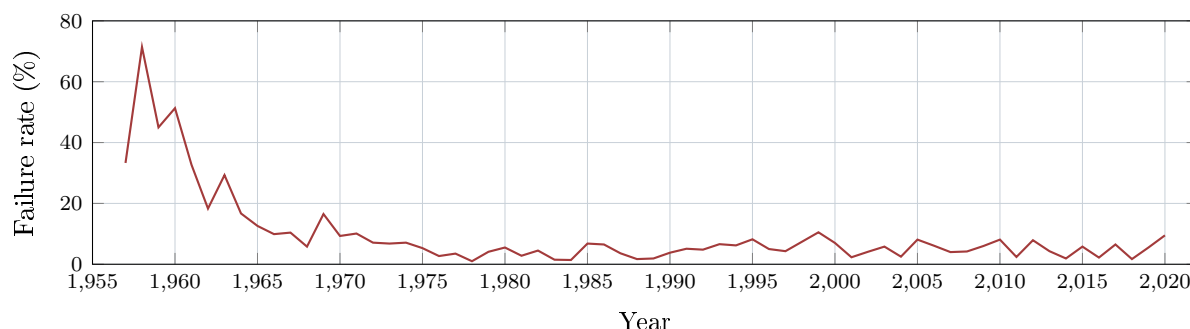


Figure 3: The project’s most defensible result: rocketry went from losing half its launches to losing a few percent, within roughly a decade. Early years are noisy because the denominators are tiny (1957 flew 3 launches). Overall: 3,879 successes against 339 failures, 102 partial and 4 prelaunch, an 89.7% success rate.

4 The four charts that overclaim

“USA vs USSR” does not compare the USA to the USSR

Charts 12 and 13 filter to two *organisations*: NASA and RVSN USSR. But RVSN USSR (the Strategic Rocket Forces) ran essentially the entire Soviet programme, while NASA flew only **203 of the 1,351** launches from American soil, which is **15%**. The rest were General Dynamics (251), the US Air Force (161), ULA (140), Boeing (136), Martin Marietta (114), SpaceX (100) and others. American spaceflight was mostly military and contractor work; NASA is the famous name, not the busy one. So the chart sets 15% of one side against 100% of the other and reports a national scoreboard.

Measured: the pie reads NASA **10.3%** over all years, and the same logic gives **5.9%** for the Cold War era. Counted by launch country over that era it is **USA 662 against RUS+KAZ 1,770**, or **27.2%**. The chart understates the American share about 4.6-fold. The USSR really did out-launch the USA roughly 2.5 to 1, which is a genuine finding; the chart claims 9 to 1. Chart 13 compounds this by running to 2020, thirty years after its subject ceased to exist. The **Country** column needed to fix this already exists in the dataframe.

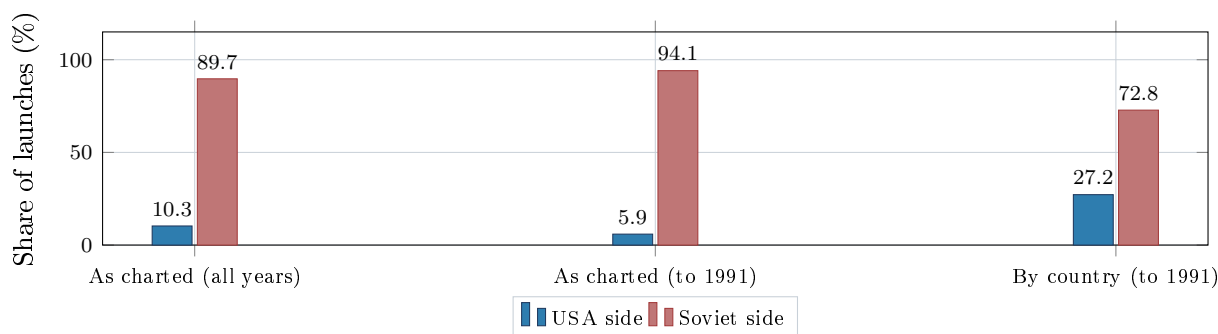


Figure 4: The size of the distortion. The first two columns use NASA as a stand-in for the USA, as the code does; the third uses the **Country** column the project already derives. All six numbers were computed from the project’s own cleaned dataframe.

The choropleth is coloured by country, not by launch count

Chart 05 is titled “Number of Launches by Country” and passes `color='Country'`, the ISO code. The count lives in a different column, handed only to `hover_data`. Since the codes are strings, Plotly treats them as categorical and *silently ignores* `color_continuous_scale='matter'`. Confirmed by building the figure: the correct call yields one trace with a colour axis, this one yields **15 separate traces** and a layout with **no colour axis**. So Russia (1,398 launches) and Brazil (3) are two arbitrary colours of equal visual weight, and the quantity appears only on hover.

The fix is one word. *app.py already does it right* (`color='Launches'`); the dashboard fixed the bug and the static script was never updated.

The price histogram shows two bars

`bins=20` spreads 20 buckets across the full price range, and prices run from 5.3 to 5,000, so each bucket is about 250 wide. `xlim(0, 1000)` then crops to the first four, of which two are non-empty. The rendered PNG was opened: two bars, and roughly 60% empty canvas. One outlier sets the bin width for all 964 priced launches, and the crop then hides the outlier that caused it.

The distribution underneath is genuinely interesting: median 62, and 98.4% of priced launches at 500 or less. A two-bar chart cannot show that. Only 964 of 4,324 launches (22%) report a price at all, and the code is right to `dropna` rather than treat missing as zero; every price conclusion describes the 22% who disclose, not the industry.

Chart 17 plots all 56 organisations and its legend covers the plot

Its comment promises “the Organisation Doing the Most Launches” year on year. The code computes no maximum; it draws 56 overlapping lines. The rendered PNG was opened: the legend needs 56 entries, grows taller than the figure, and physically covers the 1957 to 1975 region of the plot. `matplotlib` recycles its palette every 10 colours, so the legend cannot

identify anything anyway. Chart 11 is the same chart done correctly, restricted to the top 10, six lines earlier, which suggests (inferred, not proven) that 17 is a copy of 11 with the filter dropped.

Two smaller ones, for completeness. Chart 07 plots years ordered by *launch count* rather than chronologically, because a bare `value_counts()` sorts by frequency; it is a misleading near-duplicate of chart 16, which does it right. And chart 09 draws one straight line from 1973 to 1981 across eight years that contain no price data at all, because pandas plots against an index that simply omits them.

5 The dashboard

`app.py` loads the cleaned frame once into a module-level global and re-slices it in memory on every request. Nothing is precomputed per filter combination, so a filtered view is always a real pandas slice. At 4,324 rows this is instant, and the design is right: the data is far too small to justify a database.

The good decisions here are worth naming. The organisation list, the country list, the year bounds and the “Top 10” shortcut are all derived from the data rather than hardcoded, so the UI cannot drift out of sync with the CSV. Empty multi-select means “no restriction”, which matches what the labels promise. Bad year input returns HTTP 400 (verified), and the empty-selection case is guarded explicitly in both the stats builder and the figure builders rather than being left to raise `IndexError` (verified: an unknown organisation returns 200 with empty figures, not a crash). That is more care than a portfolio dashboard usually gets.

The odd-looking `json.loads(json.dumps(fig, cls=PlotlyJSONEncoder))` is a type-laundering step, not confusion: Plotly figures hold numpy types that plain `json.dumps` refuses, and Flask’s `jsonify` needs a dict rather than a pre-serialised string, so the figure is serialised by the encoder that understands it and parsed straight back into native types.

The dashboard needs the internet to draw anything

Plotly.js is loaded from a public CDN. Offline, every `Plotly.newPlot` raises `ReferenceError: Plotly is not defined` and the page renders as empty boxes with working filters and no charts, silently. For a project whose selling point is running locally against a committed CSV, that is an odd seam. The version is pinned (2.35.2), which handles the supply-chain half of the worry but not the availability half.

Separately, `index.html` hardcodes “4,324 launches, 56 organisations” in its subtitle while the stat tiles directly beneath it are rendered from the live data. The moment the CSV changes, the subtitle lies and the tile below it tells the truth.

6 Verification

Nothing here is taken on trust. A clean virtual environment was built from `requirements.txt`; the script ran to completion (exit 0, 17 files); the dashboard was started and both routes exercised; and the cleaning was replicated independently to check every count.

Claim	Result
4,324 launches, 56 organisations	confirmed
964 priced launches, about 22%	confirmed (22.3%)
25 organisations in the spend table	confirmed
17 chart files written to <code>output/</code>	confirmed , exit 0
Cold War filter: 1,876 launches, 111 NASA / 1,765 RVSN USSR	confirmed , via the live API
Committed PNGs match a fresh run	byte-identical
“drops duplicate rows”	no-op , 0 exist
“undercounts USSR-era launches”	false , 0 Soviet rows dropped
“the CSV happens to be clean”	false , mojibake in 341 values
Chart 05 colours by launch count	false , 15 categorical traces
Chart 13 compares USA to USSR	false , NASA is 15% of the USA
Chart 17 shows the yearly leader	false , plots all 56

Table 1: Every README figure checks out. The lower block is where the code, the data or the chart titles disagree with what is claimed.

The committed output is reproducible

Re-running the script regenerated all 15 PNGs byte-for-byte identically to the committed ones; only the two Plotly HTML files differed, and only in an embedded version string. The charts in `output/` are exactly what this code produces, which is a stronger guarantee than most analysis projects can offer.

On data quality, one correction to the README: the CSV is not clean. One organisation is stored as `Arm??e de l’Air` with two literal question marks where an accent belonged, and 341 `Location` and `Detail` values carry broken multi-byte sequences from the scrape. It does not affect the analysis (`Detail` is never used, and `Location` is only split on its last comma), but the README asserts the opposite and a `grep` finds it in seconds.

7 Defending it

Lead with the bugs

The strengths are real: saving every figure to disk before optionally showing it (which is what let the whole thing be verified headless), fixing the pandas `value_counts()` naming trap by mapping on keys instead of positions, dropping missing prices rather than zero-filling them, deriving the UI’s options from the data, and committing output that reproduces byte-for-byte.

But four chart titles overclaim, and each is findable in a minute by anyone reading the code. Volunteering them turns each from a liability into evidence of judgement:

1. **The USA vs USSR chart is wrong.** NASA is 15% of American launches, so it says 9:1 where the truth is about 2.5:1. Say this first; it defuses the rest.
2. **The choropleth colours by identity, not count.** One argument. `app.py` already has it right.
3. **The price histogram shows two bars** because one 5,000 outlier sets the bin width. The real distribution: median 62, 98.4% under 500.
4. **The duplicate check can never fire**, because the row-number columns are dropped after it runs. Noticing that about your own code is worth more than the check.

Then the best story here: the country lookup does not lose the USSR, it reattributes it, and files 579 Soviet launches from Baikonur under Kazakhstan. Knowing exactly how your data is wrong, rather than vaguely that it might be, is the job.

Glossary

CSV	A plain-text table: one header line naming the columns, one line per record, commas between cells. The project's input is one.
DataFrame	pandas' in-memory table, the central object of the project. A spreadsheet with named columns that you operate on a whole column at a time. One column is a <i>Series</i> .
NaN	"Not a Number", pandas' marker for a cell the source never recorded. Contagious in arithmetic, which is why missing prices must be dropped deliberately.
matplotlib	Python charting library producing static images (PNG). 15 of the 17 charts.
Plotly	Charting library producing structures a browser renders interactively (hover, zoom, click). 2 static charts, plus all 5 dashboard charts.
Flask	A small Python web framework: it routes URLs to Python functions. Runs the dashboard.
Choropleth	A map that colours each region by a value, so the colour <i>is</i> the data. Chart 05 is one, coloured by the wrong column.
Histogram	A distribution chart: chop the range into equal buckets, draw a bar per bucket showing how many records landed in it. Bucket count is a judgement call.
ISO alpha-3	A three-letter standard country code (USA , RUS , KAZ). Map libraries want codes, not names, because names are ambiguous.
pycountry	Python wrapper around the official ISO country register. Knows only <i>present-day</i> countries, which is why the USSR gets split into RUS and KAZ .
Mojibake	Text encoded in one character set and decoded as another, scrambling non-English characters. Present in this project's source CSV.